Credit Name: CSE 3130 Object-Oriented Programming 2
Assignment Name: UEmployee, Faculty, Staff

Understanding the Problem
How did you approach understanding the challenge?
Were there any parts of the problem you found confusing at first? If so, how did you resolve that confusion?

I decided to give this assignment a try before we had completed the SalesCenter example as a class primarily because I didn't know we were doing it. I did see it in the textbook, and I thought that it seemed very similar to the UEmployee project. I thought if I encountered any difficulty, I could take a look at the SalesCenter example for some guidance. UEmployee didn't seem too difficult anyways, and I did understand what I was being tasked with.

Planning the Solution
Did you create a plan or break the problem into smaller steps before coding?
How did you decide on the tools, data structures, or algorithms to use?

I figured that I would just follow along with the instructions and would make the classes as they were listed. Because of said instructions, I knew that I would need to use the *extends* and *super* keywords in order to inherit some information from the UEmployee class.

```
public class Faculty extends UEmployee
```

```
public class Staff extends UEmployee
```

```
super(name, salary);
```

Implementation
Did you write the code in small pieces or attempt the entire solution at once?
How did you test your solution along the way to make sure it was working?

As per the instructions, I started by creating the UEmployee class with local variables for the employee's name and salary. I then created the constructor and a method for returning the name of the employee.

```
public class UEmployee
{
    private String name;
    private double salary;

    public UEmployee(String n, double s)
    {
        name = n;
        salary = s;
    }

    public String getName()
    {
        return name;
    }
```

The instructions then told me to create a method for returning the salary. It was at this point where looking at the SalesCenter example may have actually been disadvantageous because in it, they used an abstract method that calculates the yearly salary and the pay rate for managers and associates respectively. However, for this assignment, only a salary was needed. I didn't really understand how the abstract method worked initially, but since it looked relevant, I implemented it anyway. It was only after I had created the Faculty and Staff classes and reread the instructions did I notice that it wasn't necessary. I replaced the abstract method with a simpler one.

```
public double getSalary()
{
    return salary;
}
```

After I had initially finished the UEmployee class, I made the Faculty class with a local variable for the department name. I inherited information from the UEmployee class using the *extends* keyword and used the *super* keyword to send information from an object to the UEmployee class.

```
private String depName;

public Faculty(String name, double salary, String depart)
{
    super(name, salary);

    depName = depart;
}
```

I finished the class by creating a method that would return the name of the employee using the getName() method from UEmployee and the department name.

```
public String toString()
{
    return(super.getName() + ", " + depName);
}
```

I basically repeated the process for the Staff class, replacing the department name with the job title.

```
public class Staff extends UEmployee
{
    private String title;

    public Staff(String name, double salary, String jobTitle)
    {
        super(name, salary);

        title = jobTitle;
    }

    public String toString()
    {
        return(super.getName() + ", " + title);
    }
}
```

The last thing I needed to do was to create a tester class to bring everything together. I decided to call it University, and most of the logic and formatting was taken directly from the SalesCenter example since it was so similar. I created the objects that would represent the employees at the university along with all the other variables I thought I would need.

```
Faculty emp1 = new Faculty("Henry", 110000, "Professor");
Faculty emp2 = new Faculty("Tony", 90000, "Associate Professor");
Staff emp3 = new Staff("Jennifer", 80000, "IT Support");
Staff emp4 = new Staff("Allan", 70000, "Secretary");

NumberFormat money = NumberFormat.getCurrencyInstance();

String choice;
int empNum;
UEmployee emp = emp1;

Scanner input = new Scanner(System.in);
```

I then recycled the logic from the SalesCenter example for the main interface that the user would interact with.

```
do
{
    System.out.println("Please Pick (E)mployee, (S)alary, or (Q)uit: ");

    choice = input.next();

    if (!choice.equalsIgnoreCase("Q"))
    {
        System.out.println("Enter Employee Number (1, 2, 3, 4): ");

        empNum = input.nextInt();

        switch (empNum)
        {
            case 1: emp = emp1; break;
            case 2: emp = emp2; break;
            case 3: emp = emp3; break;
            case 4: emp = emp4; break;
        }

        if (choice.equalsIgnoreCase("E"))
        {
            System.out.println(emp);
        }

        else if (choice.equalsIgnoreCase("S"))
        {
            System.out.println(money.format(emp.getSalary()));
        }
    }
} while (!choice.equalsIgnoreCase("Q"));

System.out.println("Goodbye!");
```

Overcoming Challenges

What part of the problem was the most difficult for you?
How did you handle moments when you felt unsure of what to do next?

The most confusing part was trying to figure out how the abstract method worked in the SalesCenter example since I thought it would be relevant. Once I figured it out by skimming through the Manager and Associate classes that were associated with the example, I realized that it wasn't even necessary.

Learning

Was there anything you learned that you think will help you with future challenges?

I think my understanding of inheritance and abstract methods in java has improved.